

TOTALTECH

Documentación de API

Referencia técnica del sistema
de venta de productos tecnológicos

81

OPERACIONES HTTP

14

MÓDULOS

JWT

AUTENTICACIÓN

Grupo 1 · Daiana Chinellato · Facundo Sola

Edición del 1 de octubre de 2026

ASP.NET Core 10 / Entity Framework Core / SQL Server

Contratos contrastados con el código actual. Ejemplos ilustrativos y fuente editable incluidas.

Contenido

01. Alcance y guía de lectura	3
02. Conexión y convenciones	4
03. Seguridad y respuestas HTTP	5
04. Catálogo paginado	6
05. Referencia de operaciones	7
06. Ejemplos de integración	16
07. Diccionario de contratos	19
08. Enumeraciones y estados	34
09. Flujo de compra y diferencias con el PDF histórico	35
10. Trazabilidad y mantenimiento	36

Lectura sugerida: conexión y seguridad para comenzar; referencia de operaciones para integrar; diccionario para revisar campos y tipos; diferencias para migrar desde la documentación anterior.

01. Alcance y guía de lectura

Esta edición adapta el PDF histórico de 54 páginas al código actual del repositorio interno. La fuente de verdad son los endpoints, DTO, entidades, lógica y repositorios inspeccionados. Los ejemplos son ilustrativos: los ID, importes y fechas no representan datos de una base real.

Validación realizada: revisión estática de contratos y cobertura de rutas, generación del PDF y revisión visual. No se ejecutaron llamadas HTTP ni pruebas contra una base de datos. La presencia de un endpoint no demuestra que exista una pantalla MVC completa para ese flujo.

El PDF original se conserva como referencia histórica. Esta edición y su fuente editable API.md constituyen la referencia vigente. El generador vuelve a extraer rutas y esquemas; las notas y ejemplos se deben revisar cuando cambie la lógica de negocio.

02. Conexión y convenciones

Elemento	Valor actual
Backend HTTP local	http://localhost:5070
Backend HTTPS local	https://localhost:7038
Rutas de recursos	Sin prefijo /api ni versión en la URL
OpenAPI en Development	/openapi/v1.json
Scalar en Development	/scalar/v1 (ruta predeterminada de MapScalarApiReference)
Cuerpos JSON	Content-Type: application/json; nombres camelCase
Identificadores	integer; parámetros de ruta con restricción int
Enumeraciones JSON	Valores numéricos; no hay JsonSerializer configurado
Importes	JSON number; no enviar números monetarios como strings
Fechas	ISO 8601; los flujos de autenticación/confirmación generan UTC

Los puertos pertenecen a Properties/launchSettings.json y pueden cambiar por configuración del entorno. No hay un host de producción documentado. OpenAPI y Scalar se mapean solo en Development y, al no tener AllowAnonymous explícito, quedan sujetos a la política de autenticación predeterminada.

El backend necesita ConnectionStrings:DefaultConnection y Authentication:Issuer, Audience y SigningKey. La clave debe tener al menos 32 bytes UTF-8; usar variables de entorno o User Secrets. Este documento no contiene credenciales. Database:ApplyMigrations controla las migraciones de inicio (por defecto habilitadas en Development); DemoData y BootstrapAdmin son opciones separadas. No se aplicaron migraciones durante esta revisión.

03. Seguridad y respuestas HTTP

Iniciar sesión mediante POST /auth/login y enviar Authorization: Bearer <accessToken> en rutas protegidas. expiresAtUtc indica el vencimiento. El token usa HS256; se verifican firma, emisor, audiencia y vigencia. ExpirationMinutes tiene valor por defecto 480 y rango 1-1440. No hay endpoint de refresh ni logout en la API.

Etiqueta	Requisito
Público	AllowAnonymous; no requiere JWT
JWT	Usuario autenticado; verificar propiedad según las notas del módulo
Admin	JWT con rol Administrador (no Admin)

Las tablas de rutas muestran los estados explícitos de cada handler. Además, las rutas protegidas pueden devolver 401 por token ausente/inválido, las administrativas 403 por rol insuficiente y el binding puede devolver 400 por JSON o tipos inválidos. Un recurso ajeno suele responder 404 para no revelar su existencia.

Estado	Significado y cuerpo
200	Lectura, actualización o repetición exitosa; JSON según recurso
201	Creación; JSON y cabecera Location
204	Éxito sin cuerpo; no intentar parsear JSON
400	Validación o binding inválido
401 / 403	Autenticación / autorización
404	Recurso inexistente o inaccesible; login usa un objeto específico
409	Relaciones que impiden eliminar, duplicado de email o conflicto de dominio

No existe un formato único de error: muchos handlers devuelven un string JSON; el catálogo usa ErrorCatalogoResponse; login inexistente devuelve {codigo, mensaje}; algunos errores y 204 no tienen cuerpo. Las listas vacías responden 200 con [], no 404. Los fallos inesperados de infraestructura no tienen contrato de error uniforme declarado.

04. Catálogo paginado

Query	Tipo y regla
texto	string opcional; trim; hasta 100 caracteres
idCategoria	integer opcional > 0
precioMin / precioMax	number opcional >= 0; mínimo <= máximo
soloDisponibles	boolean; por defecto false
pagina	integer >= 1; por defecto 1
tamanoPagina	integer de 1 a 48; por defecto 12

Ejemplo: GET /productos/catalogo?texto=notebook&soloDisponibles=true&pagina=1&tamanoPagina=12. Error de filtro: 400 con codigo=catalogo_parametros_invalidos y mensaje. Respuesta: items, pagina, tamanoPagina, totalItems y totalPaginas.

05. Referencia de operaciones

Los nombres de body remiten al diccionario de contratos. El guion indica que no se recibe cuerpo JSON. Los parámetros entre llaves pertenecen a la ruta; los query se explican en las notas. GET devuelve una colección en las rutas de listado/filtro y un objeto en las rutas por ID.

Autenticación

Identidad, registro público y emisión de tokens JWT.

Método	Ruta	Acceso	Body	HTTP
POST	/auth/login	Público	LoginDto	200/401/404
POST	/auth/registro	Público	UsuarioRequest	201/400/409
POST	/auth/recuperar-contrasena	Público	RecuperarContrasenaDto	200

- POST /auth/login devuelve LoginResponse: datos de UsuarioResponse, accessToken y expiresAtUtc. Email inexistente: 404 con codigo=usuario_no_registrado y mensaje; contraseña incorrecta: 401 sin cuerpo específico.
- POST /auth/registro devuelve UsuarioResponse y Location=/usuarios/{idUsuario}. El servidor fuerza rol=0 (Cliente) y fechaRegistro UTC. Nombre, apellido, email válido y teléfono son obligatorios; contraseña de al menos 8 caracteres; email duplicado: 409.
- POST /auth/recuperar-contrasena recibe email y devuelve {mensaje}. La implementación solo consulta si existe el usuario; no registra una solicitud persistente, no envía correo y no emite un token de recuperación.

Fuente: Totaltech/Endpoints/AuthEndpoints.cs y lógica/repositorios del módulo.

Productos

Catálogo público y mantenimiento administrativo de productos.

Método	Ruta	Acceso	Body	HTTP
GET	/productos/catalogo	Público	-	200/400
GET	/productos	Público	-	200
GET	/productos/{id}	Público	-	200/404
POST	/productos	Admin	ProductoRequest	201/400
PUT	/productos/{id}	Admin	ProductoRequest	200/400/404
DELETE	/productos/{id}	Admin	-	204/404/409
GET	/productos/buscar	Público	-	200
GET	/productos/categoria/{idCategoria}	Público	-	200
GET	/productos/disponibles	Público	-	200
PATCH	/productos/{id}/stock	Admin	ActualizarStockRequest	204/400/404

- GET /productos/catalogo devuelve CatalogoProductosResponse con items de ProductoCatalogoResponse. Es la ruta paginada. Las otras rutas GET devuelven Producto o listas de Producto sin paginación.
- GET /productos/buscar recibe texto opcional. GET /productos/categoria/{idCategoria} filtra por categoría; /disponibles filtra stock positivo.
- POST y PUT devuelven Producto; POST incluye Location. nombre obligatorio, precio y stock ≥ 0 , categoría y proveedor existentes. PATCH /{id}/stock recibe el stock absoluto y devuelve 204.
- imageUrl es nullable. En PUT, omitirla o enviar null conserva la imagen vigente. La API recibe una ruta JSON, no un archivo multipart. Solo admite /images/categorias/... o /uploads/productos/{GUID de 32 caracteres}.jpg|png|webp, hasta 500 caracteres; no admite URLs externas ni segmentos de traversal.
- La carga de archivos ocurre en Frontend mediante ProductoImagenStorage (JPG, PNG o WebP, hasta 5 MB), con almacenamiento en wwwroot/uploads/productos. No existe una ruta de subida de imágenes en esta API.

Fuente: Totaltech/Endpoints/ProductosEndpoints.cs y lógica/repositorios del módulo.

Categorías

Clasificación pública del catálogo; escritura administrativa.

Método	Ruta	Acceso	Body	HTTP
GET	/categorias	Público	-	200
GET	/categorias/{id}	Público	-	200/404
POST	/categorias	Admin	CategoriaRequest	201/400
PUT	/categorias/{id}	Admin	CategoriaRequest	200/400/404
DELETE	/categorias/{id}	Admin	-	204/404/409

- GET devuelve Categoría o listas; POST y PUT devuelven Categoría. nombre es obligatorio. DELETE exitoso no devuelve cuerpo.

Fuente: Totaltech/Endpoints/CategoriasEndpoints.cs y lógica/repositorios del módulo.

Carritos

Carrito activo del usuario autenticado y confirmación de compra.

Método	Ruta	Acceso	Body	HTTP
GET	/carritos/actual	JWT	-	200/401
POST	/carritos/actual/productos	JWT	AgregarProductoCarritoDto	200/400/401/409
PATCH	/carritos/actual/productos/{idProducto}	JWT	ActualizarCantidadCarritoDto	200/400/401/409
DELETE	/carritos/actual/productos/{idProducto}	JWT	-	200/401/404
GET	/carritos	JWT	-	200/401
GET	/carritos/{id}	JWT	-	200/404
POST	/carritos	JWT	CarritoRequest	201/400
PUT	/carritos/{id}	JWT	CarritoRequest	200/400/404
DELETE	/carritos/{id}	JWT	-	204/404/409
GET	/carritos/usuario/{idUsuario}	JWT	-	200/404
POST	/carritos/{idCarrito}/productos	JWT	AgregarProductoCarritoDto	201/400/404
DELETE	/carritos/{idCarrito}/productos/{idProducto}	JWT	-	204/400/404
POST	/carritos/{idCarrito}/confirmar	JWT	ConfirmarCarritoDto	200/201/400/404/ 409

- Las cuatro rutas /actual trabajan siempre sobre el ID del JWT y devuelven CarritoResumenResponse. GET sin carrito activo devuelve idCarrito=null, items=[], cantidadTotal=0 y total=0; no crea uno por consultar.
- POST /actual/productos suma la cantidad a una línea existente o crea la línea y el carrito cuando corresponde. PATCH fija la cantidad absoluta (> 0); no elimina con cero. DELETE devuelve el resumen actualizado (200), o 404 si la línea no existe.
- El precioUnitario del AgregarProductoCarritoDto se ignora: el servidor consulta el precio vigente del producto. Stock insuficiente o conflicto concurrente en POST/PATCH /actual: 409. Entrada inválida: 400.
- Las rutas CRUD devuelven Carrito o listas. GET / y /usuario/{idUsuario} limitan clientes a sus propios carritos; el administrador puede acceder a los de otros. GET/PUT/DELETE por ID requieren propiedad o rol administrador.
- En POST /carritos, para clientes el servidor fija idUsuario, fechaCreacion UTC y estado=Activo. En PUT conserva esos tres valores para clientes. En administradores se utilizan los valores del request.
- POST /{idCarrito}/productos devuelve DetalleCarrito y 201; DELETE /{idCarrito}/productos/{idProducto} devuelve 204. Ambos requieren carrito propio o acceso administrador. Usar las rutas /actual para modificar cantidades desde el catálogo.
- POST /{idCarrito}/confirmar recibe únicamente idDireccion. Devuelve PedidoResponse: 201 al crear; 200 al repetir con la misma dirección; 409 si ya se confirmó con otra dirección, estado incompatible o stock insuficiente; 404 si faltan recursos; 400 para dirección inválida o carrito vacío.

- La confirmación exige dirección perteneciente al usuario del carrito; calcula precios y total, descuenta stock y guarda una copia de la dirección y líneas del pedido en una transacción relacional. No registra ni aprueba un pago.

Fuente: Totaltech/Endpoints/CarritosEndpoints.cs y lógica/repositorios del módulo.

Pedidos

Consulta de pedidos y operaciones de estado y pago.

Método	Ruta	Acceso	Body	HTTP
GET	/pedidos	JWT	-	200/401
GET	/pedidos/{id}	JWT	-	200/404
GET	/pedidos/{id}/detalles	JWT	-	200/404
GET	/pedidos/usuario/{idUsuario}	JWT	-	200/404
GET	/pedidos/estado/{estado}	Admin	-	200/400
PATCH	/pedidos/{id}/estado	Admin	ActualizarEstadoPedidoRequest	204/400/404/409
POST	/pedidos/{idPedido}/pagos	Admin	CrearPagoParaPedidoRequest	201/400
GET	/pedidos/{idPedido}/pagos	JWT	-	200/404

- GET / y /usuario/{idUsuario} devuelven listas de PedidoResponse filtradas por propiedad para clientes. GET /{id} devuelve PedidoResponse. El administrador puede acceder a todos; /estado/{estado} es exclusivamente administrativo.
- PedidoResponse.detalles se devuelve vacío por el mapper actual en las rutas de pedido y confirmación. Obtener las líneas reales mediante GET /pedidos/{id}/detalles, que devuelve DetallePedidoResponse[]. La dirección es una copia histórica, no una navegación EF.
- PATCH /{id}/estado recibe ActualizarEstadoPedidoRequest. Transiciones: Pendiente -> Cancelado (restituye stock), Pagado -> Enviado, Enviado -> Entregado. Repetir el mismo estado es exitoso. Pendiente -> Pagado ocurre por aprobación de pagos, no por este PATCH.
- POST /{idPedido}/pagos es administrativo y devuelve PagoResponse (201). Solo admite pedidos pendientes, monto > 0 y metodoPago válido; estado inicial Pendiente y fecha UTC si no se envía. GET /{idPedido}/pagos devuelve PagoResponse[] con control de propiedad.
- No existen POST /pedidos, PUT /pedidos/{id} ni DELETE /pedidos/{id}. Los pedidos se crean al confirmar el carrito.

Fuente: Totaltech/Endpoints/PedidosEndpoints.cs y lógica/repositorios del módulo.

Pagos

Consulta y transición controlada de pagos registrados.

Método	Ruta	Acceso	Body	HTTP
GET	/pagos	Admin	-	200
GET	/pagos/{id}	JWT	-	200/404
PATCH	/pagos/{id}/estado	Admin	ActualizarEstadoPagoRequest	200/400/404/409

- GET / devuelve PagoResponse[] y requiere administrador; GET /{id} devuelve PagoResponse y requiere propiedad del pedido o administrador.
- PATCH /{id}/estado es administrativo y devuelve PagoResponse (200). Un pago Pendiente puede pasar a Aprobado, Rechazado o Cancelado. Repetir el mismo estado es exitoso; otras transiciones: 409.
- Aprobar exige pedido Pendiente. La suma de pagos aprobados no puede superar el total; al igualarlo, el pedido pasa a Pagado. No se implementa una pasarela ni un webhook: MetodoPago identifica el método del registro.
- No existen POST /pagos, PUT /pagos/{id} ni DELETE /pagos/{id}; crear mediante POST /pedidos/{idPedido}/pagos.

Fuente: Totaltech/Endpoints/PagosEndpoints.cs y lógica/repositorios del módulo.

Detalles de pedidos

Lectura de líneas históricas generadas al confirmar un carrito.

Método	Ruta	Acceso	Body	HTTP
GET	/detallepedidos	Admin	-	200
GET	/detallepedidos/{id}	JWT	-	200/404

- GET / devuelve DetallePedidoResponse[] solo para administrador. GET /{id} devuelve DetallePedidoResponse si el pedido es propio o el usuario es administrador.
- DetallePedidoResponse no incluye idPedido ni propiedades de navegación. No existen POST, PUT ni DELETE para este recurso. Las líneas se generan al confirmar el carrito; precio y subtotal quedan fijados.

Fuente: Totaltech/Endpoints/DetallePedidosEndpoints.cs y lógica/repositorios del módulo.

Detalles de carritos

Lectura de líneas; las modificaciones se realizan mediante /carritos.

Método	Ruta	Acceso	Body	HTTP
GET	/detallecarritos	JWT	-	200/401
GET	/detallecarritos/{id}	JWT	-	200/404
GET	/detallecarritos/carrito/{idCarrito}	JWT	-	200/404

- Las respuestas son DetalleCarrito o listas. GET / devuelve todas las líneas al administrador y solo líneas de carritos propios al cliente. GET por ID o por carrito exige propiedad o administrador.
- No existen POST, PUT ni DELETE /detallecarritos. Agregar, cambiar cantidad y eliminar productos mediante /carritos/actual/productos o las rutas por carrito.

Fuente: Totaltech/Endpoints/DetalleCarritosEndpoints.cs y lógica/repositorios del módulo.

Usuarios

Administración y edición del perfil propio sin exponer contraseñas.

Método	Ruta	Acceso	Body	HTTP
GET	/usuarios	Admin	-	200
GET	/usuarios/{id}	JWT	-	200/404
POST	/usuarios	Admin	UsuarioRequest	201/400/409
PUT	/usuarios/{id}	JWT	UsuarioRequest	200/400/404/409
DELETE	/usuarios/{id}	JWT	-	204/404/409

- GET / y POST / requieren administrador. GET, PUT y DELETE /{id} permiten al propietario o administrador. Todas las respuestas con cuerpo usan UsuarioResponse, sin contraseña ni hash.
- POST crea con UsuarioRequest y devuelve 201; PUT devuelve UsuarioResponse (200). Para clientes, PUT conserva el rol y fecha de registro existentes. En actualización, contraseña vacía conserva la vigente; si se cambia, debe tener al menos 8 caracteres.
- El identificador es idUsuario; no existe direccionID en UsuarioRequest. Las direcciones se relacionan desde Direccion.idUsuario. Email duplicado: 409; validación: 400.

Fuente: Totaltech/Endpoints/UsuariosEndpoints.cs y lógica/repositorios del módulo.

Direcciones

Direcciones de usuarios y control de propiedad.

Método	Ruta	Acceso	Body	HTTP
GET	/direcciones	JWT	-	200/401
GET	/direcciones/{id}	JWT	-	200/404
POST	/direcciones	JWT	DireccionRequest	201/400
PUT	/direcciones/{id}	JWT	DireccionRequest	200/400/404
DELETE	/direcciones/{id}	JWT	-	204/404/409

- GET / devuelve Direccion[]: las propias para clientes y todas para administradores. GET/PUT/DELETE por ID exigen propietario o administrador. POST y PUT devuelven Direccion.
- numero es string, no integer. calle y numero obligatorios; tipo debe ser válido y el usuario indicado debe existir. Para clientes, POST y PUT reemplazan idUsuario por el ID del JWT.

Fuente: Totaltech/Endpoints/DireccionesEndpoints.cs y lógica/repositorios del módulo.

Proveedores

Información comercial y dirección fiscal del proveedor.

Método	Ruta	Acceso	Body	HTTP
GET	/proveedores	Admin	-	200
GET	/proveedores/{id}	Admin	-	200/404
POST	/proveedores	Admin	ProveedorRequest	201/400
PUT	/proveedores/{id}	Admin	ProveedorRequest	200/400/404
DELETE	/proveedores/{id}	Admin	-	204/404/409

- Todas las operaciones requieren administrador. GET devuelve Proveedor o listas; POST/PUT devuelven Proveedor. El contrato actual usa razonSocial, cuit, emailComercial, telefonoComercial, condicionIva, direccion anidada, plazos, monedaPreferida y activo.

- Obligatorios: razón social, CUIT (máximo 20 caracteres), email comercial, condición IVA, moneda y dirección con calle/numero. Plazos no negativos. La dirección fiscal no tiene usuario asociado. No se documenta validación de dígito verificador del CUIT ni 409 para duplicados porque la lógica no los implementa.

Fuente: Totaltech/Endpoints/ProveedoresEndpoints.cs y lógica/repositorios del módulo.

Compras a proveedores

Registros de compras a proveedores; recurso distinto de los pedidos de clientes.

Método	Ruta	Acceso	Body	HTTP
GET	/compras	Admin	-	200
GET	/compras/{id}	Admin	-	200/404
POST	/compras	Admin	CompraRequest	201/400
PUT	/compras/{id}	Admin	CompraRequest	200/400/404
DELETE	/compras/{id}	Admin	-	204/404/409

- Todas las operaciones requieren administrador. GET devuelve Compra o listas; POST/PUT devuelven Compra. idProveedor debe existir, total ≥ 0 y estado válido. La lógica asigna fecha actual si se omite.

- El recurso registra datos generales de compras a proveedores. No representa el checkout del cliente ni agrega automáticamente líneas de reposición o stock.

Fuente: Totaltech/Endpoints/ComprasEndpoints.cs y lógica/repositorios del módulo.

Reportes

Registros de reportes y agregados de ventas e ingresos.

Método	Ruta	Acceso	Body	HTTP
GET	/reportes	Admin	-	200
GET	/reportes/{id}	Admin	-	200/404
POST	/reportes	Admin	ReporteRequest	201/400
PUT	/reportes/{id}	Admin	ReporteRequest	200/400/404
DELETE	/reportes/{id}	Admin	-	204/404/409
GET	/reportes/ventas	Admin	-	200
GET	/reportes/ingresos	Admin	-	200
GET	/reportes/productos-mas-vendidos	Admin	-	200

- CRUD administrativo: devuelve Reporte o listas, recibe ReporteRequest. tipoReporte válido, usuario existente y fechaInicio <= fechaFin cuando ambas estén informadas.
- GET /ventas devuelve ReporteVentasDto: cantidadPedidos y totalVentas. Incluye pedidos Pagado, Enviado o Entregado y suma subtotales de sus detalles.
- GET /ingresos devuelve ReporteIngresosDto: cantidadPagosAprobados y totalIngresos. Solo cuenta pagos Aprobado.
- GET /productos-mas-vendidos devuelve ProductoMasVendidoDto[], hasta 10 productos ordenados por cantidadVendida en pedidos Pagado, Enviado o Entregado.
- Los tres agregados no reciben filtros de fechas. Los registros CRUD de Reporte no aplican automáticamente su período a estos agregados.

Fuente: Totaltech/Endpoints/ReportesEndpoints.cs y lógica/repositorios del módulo.

Consultas

Contacto público y gestión administrativa de consultas.

Método	Ruta	Acceso	Body	HTTP
GET	/consultas	Admin	-	200
GET	/consultas/{id}	Admin	-	200/404
GET	/consultas/usuario/{idUser}	JWT	-	200/404
POST	/consultas	Público	ConsultaRequest	201/400
PUT	/consultas/{id}	Admin	ConsultaRequest	200/400/404
DELETE	/consultas/{id}	Admin	-	204/404/409

- POST es público y devuelve Consulta (201). El servidor fija estado=Pendiente y fechaConsulta UTC; idUsuario se toma del JWT si está autenticado o queda null para visitantes. Los valores equivalentes del request no controlan esos campos.
- nombre, email válido y mensaje son obligatorios. GET /, GET /{id}, PUT y DELETE son administrativos. GET /usuario/{idUsuario} permite propietario o administrador y devuelve Consulta[].
- El recurso almacena consultas y su estado; no implementa por sí mismo envío de respuestas por email.

Fuente: Totaltech/Endpoints/ConsultasEndpoints.cs y lógica/repositorios del módulo.

06. Ejemplos de integración

Los requests siguientes son JSON válido. Sustituir ID por recursos existentes. Las respuestas muestran valores ilustrativos y no son resultados obtenidos por HTTP.

Registro público

POST /auth/registro

```
{
  "nombre": "Ana",
  "apellido": "Pérez",
  "email": "ana@example.com",
  "contrasena": "<contraseña de al menos 8 caracteres>",
  "telefono": "3493000000"
}
```

Respuesta 201: UsuarioResponse (idUserio, nombre, apellido, email, telefono, fechaRegistro, rol=0); el registro no emite un token. Ejecutar login a continuación.

Inicio de sesión

POST /auth/login

```
{
  "email": "ana@example.com",
  "contrasena": "<contraseña elegida>"
}
```

Respuesta 200 ilustrativa:

```
{
  "idUserio": 7,
  "nombre": "Ana",
  "apellido": "Pérez",
  "email": "ana@example.com",
  "telefono": "3493000000",
  "fechaRegistro": "2026-10-01T12:00:00Z",
  "rol": 0,
  "accessToken": "<JWT emitido por el servidor>",
  "expiresAtUtc": "2026-10-01T20:00:00Z"
}
```

Producto y stock (administrador)

POST /productos o PUT /productos/{id}

```
{
  "nombre": "Notebook de ejemplo",
  "descripcion": "Equipo para oficina",
  "imagenUrl": null,
  "precio": 850000,
  "stock": 10,
  "idCategoria": 1,
  "idProveedor": 1
}
```

PATCH /productos/{id}/stock recibe {"stock": 8} y devuelve 204. En PUT, imagenUrl=null conserva la imagen existente.

Dirección de envío

POST /direcciones (JWT del cliente)

```
{
  "calle": "Av. Independencia",
  "numero": "123",
  "ciudad": "Sunchales",
  "provincia": "Santa Fe",
  "codigoPostal": "2322",
  "pais": "Argentina",
  "tipo": 0
}
```

Respuesta 201: Direccion con idDireccion e idUsuario asignado desde el JWT; usar idDireccion en la confirmación.

Agregar al carrito activo

POST /carritos/actual/productos (JWT del cliente)

```
{"idProducto": 1, "cantidad": 2}
```

Respuesta 200 ilustrativa:

```
{
  "idCarrito": 3,
  "items": [
    {
      "idProducto": 1,
      "nombre": "Notebook de ejemplo",
      "descripcion": "Equipo para oficina",
      "precioUnitario": 850000,
      "cantidad": 2,
      "subtotal": 1700000
    }
  ],
  "cantidadTotal": 2,
  "total": 1700000
}
```

PATCH /carritos/actual/productos/1 recibe {"cantidad": 3}. DELETE en la misma ruta elimina la línea y devuelve el resumen actualizado.

Confirmación y consulta de líneas

POST /carritos/3/confirmar

```
{"idDireccion": 5}
```

Respuesta 201 ilustrativa:

```
{
  "idPedido": 12,
  "idCarrito": 3,
  "idUsuario": 7,
  "fechaPedido": "2026-10-01T12:30:00Z",
  "estado": 0,
  "total": 1700000,
  "idDireccion": 5,
  "direccion": {
    "calle": "Av. Independencia",
    "numero": "123",
    "ciudad": "Sunchales",
    "provincia": "Santa Fe",
    "codigoPostal": "2322",
    "pais": "Argentina"
  },
  "detalles": []
}
```

GET /pedidos/12/detalles devuelve las líneas; ejemplo: [{"idDetallePedido": 20, "idProducto": 1, "cantidad": 2, "precioUnitario": 850000, "subtotal": 1700000}]. Repetir la confirmación con dirección 5 devuelve 200 con el mismo pedido.

Registro y aprobación de pago

POST /pedidos/12/pagos (JWT administrador)

```
{"metodoPago": 2, "monto": 1700000}
```

Respuesta 201: PagoResponse con estado=0. PATCH /pagos/{idPago}/estado recibe {"estado": 1} y devuelve el pago aprobado. Si la suma aprobada alcanza el total, el pedido queda Pagado. Luego PATCH /pedidos/12/estado con {"estado": 2} permite enviarlo y con {"estado": 3} entregarlo.

Consulta pública

POST /consultas

```
{
  "nombre": "Visitante",
  "email": "visitante@example.com",
  "mensaje": "Quisiera información sobre envíos."
}
```

Respuesta 201: Consulta con idConsulta, idUsuario=null si no hay identidad autenticada, fechaConsulta UTC y estado=0.

07. Diccionario de contratos

Los campos listados se extraen de las propiedades públicas del código. Nullable indica capacidad del tipo de aceptar null; no sustituye la validación de negocio. Los DTO de entrada no contienen clave primaria. En PUT enviar todos los campos que se desean conservar, salvo las excepciones expresamente indicadas.

Los contratos de respuesta dedicados son preferibles para integración. Varios recursos todavía serializan entidades EF: sus propiedades de navegación pueden aparecer como objetos o null según la consulta; no asumir que siempre son null ni que están siempre cargadas.

ActualizarCantidadCarritoDto

Campo JSON	Tipo
cantidad	integer

Fuente: Totaltech/Logica/DTOs/CarritoResumenDtos.cs

ActualizarEstadoPagoRequest

Campo JSON	Tipo
estado	integer (EstadoPago)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

ActualizarEstadoPedidoRequest

Campo JSON	Tipo
estado	integer (EstadoPedido)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

ActualizarStockRequest

Campo JSON	Tipo
stock	integer

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

AgregarProductoCarritoDto

Campo JSON	Tipo
idProducto	integer
cantidad	integer
precioUnitario	number

Fuente: Totaltech/Logica/DTOs/AgregarProductoCarritoDto.cs

CarritoRequest

Campo JSON	Tipo
idUsuario	integer
fechaCreacion	string (fecha ISO 8601)
estado	integer (EstadoCarrito)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

CategoriaRequest

Campo JSON	Tipo
nombre	string
descripcion	string

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

CompraRequest

Campo JSON	Tipo
idProveedor	integer
fechaCompra	string (fecha ISO 8601)
total	number
estado	integer (EstadoCompra)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

ConfirmarCarritoDto

Campo JSON	Tipo
idDireccion	integer

Fuente: Totaltech/Logica/DTOs/ConfirmarCarritoDto.cs

ConsultaRequest

Campo JSON	Tipo
idUsuario	integer o null
nombre	string
email	string
mensaje	string
fechaConsulta	string (fecha ISO 8601)
estado	integer (EstadoConsulta)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

CrearPagoParaPedidoRequest

Campo JSON	Tipo
fechaPago	string (fecha ISO 8601) o null
metodoPago	integer (MetodoPago)
monto	number

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

DireccionProveedorRequest

Campo JSON	Tipo
calle	string
numero	string
ciudad	string o null
provincia	string o null
codigoPostal	string o null
pais	string o null

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

DireccionRequest

Campo JSON	Tipo
idUsuario	integer o null
calle	string
numero	string
ciudad	string
provincia	string
codigoPostal	string
pais	string
tipo	integer (TipoDireccion)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

LoginDto

Campo JSON	Tipo
email	string
contrasena	string

Fuente: Totaltech/Logica/DTOs/LoginDto.cs

ProductoRequest

Campo JSON	Tipo
nombre	string
descripcion	string
imagenUrl	string o null
precio	number
stock	integer
idCategoria	integer
idProveedor	integer

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

ProveedorRequest

Campo JSON	Tipo
razonSocial	string
cuit	string
emailComercial	string
telefonoComercial	string
condicionIva	string
direccion	DireccionProveedorRequest o null
plazoPagoDias	integer
tiempoEntregaDias	integer
monedaPreferida	string
activo	boolean

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

RecuperarContraseñaDto

Campo JSON	Tipo
email	string

Fuente: Totaltech/Logica/DTOs/RecuperarContraseñaDto.cs

ReporteRequest

Campo JSON	Tipo
tipoReporte	integer (TipoReporte)
fechaInicio	string (fecha ISO 8601)
fechaFin	string (fecha ISO 8601)
idUsuario	integer

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

UsuarioRequest

Campo JSON	Tipo
nombre	string
apellido	string
email	string
contrasena	string
telefono	string
fechaRegistro	string (fecha ISO 8601)
rol	integer (RolUsuario)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

UsuarioResponse

Campo JSON	Tipo
idUsuario	integer
nombre	string
apellido	string
email	string
telefono	string
fechaRegistro	string (fecha ISO 8601)
rol	integer (RolUsuario)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

LoginResponse

Campo JSON	Tipo
idUsuario	integer
nombre	string
apellido	string
email	string
telefono	string
fechaRegistro	string (fecha ISO 8601)
rol	integer (RolUsuario)
accessToken	string
expiresAtUtc	string (fecha ISO 8601)

Fuente: Totaltech/Logica/DTOs/CrudDtos.cs

ProductoCatalogoResponse

Campo JSON	Tipo
idProducto	integer
nombre	string
descripcion	string
imagenUrl	string o null
precio	number
stock	integer
idCategoria	integer
categoriaNombre	string
idProveedor	integer
proveedorNombre	string

Fuente: Totaltech/Logica/DTOs/CatalogoProductosDtos.cs

CatalogoProductosResponse

Campo JSON	Tipo
items	array de ProductoCatalogoResponse
pagina	integer
tamanoPagina	integer
totalItems	integer
totalPaginas	integer

Fuente: Totaltech/Logica/DTOs/CatalogoProductosDtos.cs

ErrorCatalogoResponse

Campo JSON	Tipo
codigo	string
mensaje	string

Fuente: Totaltech/Logica/DTOs/CatalogoProductosDtos.cs

LineaCarritoResponse

Campo JSON	Tipo
idProducto	integer
nombre	string
descripcion	string
precioUnitario	number
cantidad	integer
subtotal	number

Fuente: Totaltech/Logica/DTOs/CarritoResumenDtos.cs

CarritoResumenResponse

Campo JSON	Tipo
idCarrito	integer o null
items	array de LineaCarritoResponse
cantidadTotal	integer
total	number

Fuente: Totaltech/Logica/DTOs/CarritoResumenDtos.cs

DireccionPedidoResponse

Campo JSON	Tipo
calle	string
numero	string
ciudad	string
provincia	string
codigoPostal	string
pais	string

Fuente: Totaltech/Logica/DTOs/PedidosDtos.cs

DetallePedidoResponse

Campo JSON	Tipo
idDetallePedido	integer
idProducto	integer
cantidad	integer
precioUnitario	number
subtotal	number

Fuente: Totaltech/Logica/DTOs/PedidosDtos.cs

PedidoResponse

Campo JSON	Tipo
idPedido	integer
idCarrito	integer o null
idUsuario	integer o null
fechaPedido	string (fecha ISO 8601)
estado	integer (EstadoPedido)
total	number
idDireccion	integer
direccion	DireccionPedidoResponse
detalles	array de DetallePedidoResponse

Fuente: Totaltech/Logica/DTOs/PedidosDtos.cs

PagoResponse

Campo JSON	Tipo
idPago	integer
idPedido	integer
fechaPago	string (fecha ISO 8601)
metodoPago	integer (MetodoPago)
monto	number
estado	integer (EstadoPago)

Fuente: Totaltech/Logica/DTOs/PedidosDtos.cs

ReporteVentasDto

Campo JSON	Tipo
cantidadPedidos	integer
totalVentas	number

Fuente: Totaltech/Logica/DTOs/ReporteVentasDto.cs

ReporteIngresosDto

Campo JSON	Tipo
cantidadPagosAprobados	integer
totalIngresos	number

Fuente: Totaltech/Logica/DTOs/ReporteIngresosDto.cs

ProductoMasVendidoDto

Campo JSON	Tipo
idProducto	integer
nombre	string
cantidadVendida	integer
totalVendido	number

Fuente: Totaltech/Logica/DTOs/ProductoMasVendidoDto.cs

Producto

Campo JSON	Tipo
idProducto	integer
nombre	string
descripcion	string
imagenUrl	string o null
precio	number
stock	integer
idCategoria	integer
idProveedor	integer
categoria	Categoria o null
proveedor	Proveedor o null

Fuente: Totaltech/Entidades/Producto.cs

Categoría

Campo JSON	Tipo
idCategoria	integer
nombre	string
descripcion	string

Fuente: Totaltech/Entidades/Categoria.cs

Carrito

Campo JSON	Tipo
idCarrito	integer
idUsuario	integer
fechaCreacion	string (fecha ISO 8601)
estado	integer (EstadoCarrito)
usuario	Usuario o null

Fuente: Totaltech/Entidades/Carrito.cs

DetalleCarrito

Campo JSON	Tipo
idDetalleCarrito	integer
idCarrito	integer
idProducto	integer
cantidad	integer
precioUnitario	number
subtotal	number
carrito	Carrito o null
producto	Producto o null

Fuente: Totaltech/Entidades/DetalleCarrito.cs

Direccion

Campo JSON	Tipo
idDireccion	integer
idUsuario	integer o null
usuario	Usuario o null
calle	string
numero	string
ciudad	string
provincia	string
codigoPostal	string
pais	string
tipo	integer (TipoDireccion)

Fuente: Totaltech/Entidades/Direccion.cs

Proveedor

Campo JSON	Tipo
idProveedor	integer
razonSocial	string
cuit	string
emailComercial	string
telefonoComercial	string
condicionIva	string
idDireccion	integer o null
plazoPagoDias	integer
tiempoEntregaDias	integer
monedaPreferida	string
activo	boolean
direccion	Direccion o null

Fuente: Totaltech/Entidades/Proveedor.cs

Compra

Campo JSON	Tipo
idCompra	integer
idProveedor	integer
fechaCompra	string (fecha ISO 8601)
total	number
estado	integer (EstadoCompra)
proveedor	Proveedor o null

Fuente: Totaltech/Entidades/Compra.cs

Reporte

Campo JSON	Tipo
idReporte	integer
tipoReporte	integer (TipoReporte)
fechaInicio	string (fecha ISO 8601)
fechaFin	string (fecha ISO 8601)
idUsuario	integer
usuario	Usuario o null

Fuente: Totaltech/Entidades/Reporte.cs

Consulta

Campo JSON	Tipo
idConsulta	integer
idUsuario	integer o null
nombre	string
email	string
mensaje	string
fechaConsulta	string (fecha ISO 8601)
estado	integer (EstadoConsulta)
usuario	Usuario o null

Fuente: Totaltech/Entidades/Consulta.cs

08. Enumeraciones y estados

Tipo	Valores JSON
EstadoCarrito	0=Activo; 1=Confirmado; 2=Cancelado
EstadoCompra	0=Pendiente; 1=Confirmada; 2=Cancelada
EstadoConsulta	0=Pendiente; 1=Respondida; 2=Cerrada
TipoDireccion	0=Envio; 1=Facturacion; 2=Fiscal
MetodoPago	0=Tarjeta; 1=MercadoPago; 2=Transferencia; 3=Efectivo
EstadoPago	0=Pendiente; 1=Aprobado; 2=Rechazado; 3=Cancelado
EstadoPedido	0=Pendiente; 1=Pagado; 2=Enviado; 3=Entregado; 4=Cancelado
TipoReporte	0=Ventas; 1=Compras; 2=Usuarios
RolUsuario	0=Cliente; 1=Administrador

En rutas como GET /pedidos/estado/{estado}, usar el valor numérico (por ejemplo /pedidos/estado/1). Los estados de pedidos y pagos no se pueden cambiar libremente: respetar las transiciones de sus módulos.

09. Flujo de compra y diferencias con el PDF histórico

- 1. Registrar usuario y obtener JWT mediante login.
- 2. Consultar /productos/catalogo y agregar productos al carrito /actual.
- 3. Crear o elegir una dirección propia mediante /direcciones.
- 4. Confirmar /carritos/{idCarrito}/confirmar; conservar el idPedido y consultar sus detalles.
- 5. Un administrador registra y aprueba el pago desde /pedidos/{idPedido}/pagos y /pagos/{idPago}/estado.
- 6. Un administrador cambia el pedido Pagado a Enviado y luego Entregado.

Documento histórico	Estado actual verificado
Login sin JWT; email inexistente 401	JWT con vencimiento; email inexistente 404
rol, estados y métodos como strings	Enumeraciones numéricas en JSON
id genérico, categorialID, proveedorID, usuarioID	idUsuario, idProducto, idCategoria, idProveedor y claves específicas
Proveedor con nombreEmpresa y contacto	Razón social, CUIT, condiciones y dirección fiscal anidada
CRUD libre de pedidos, pagos y detalles	Creación desde carrito/pedido y cambios de estado controlados
DELETE con mensaje o 200	Normalmente 204; eliminar línea de /actual devuelve resumen 200
Catálogo sin paginación e imágenes	/productos/catalogo paginado e imageUrl local
Sin permisos detallados	Público, JWT, administrador y propiedad por recurso
Recuperación descrita como solicitud registrada	Mensaje genérico; sin persistencia, envío ni reset implementados

10. Trazabilidad y mantenimiento

Fuentes principales: Totaltech/Program.cs; Totaltech/Endpoints/*.cs; Totaltech/Seguridad/*.cs; Totaltech/Logica/DTOs/*.cs; Totaltech/Logica/*Logica.cs; Totaltech/Repositorios/*Repositorio.cs; Totaltech/Entidades/*.cs; Totaltech/Validaciones/ProductoImagenUrl.cs; Frontend/Services/ProductoImagenStorage.cs y controladores de producto.

El alcance documental no modifica endpoints, permisos, dependencias, reglas de negocio ni esquema. El frontend contiene módulos todavía reservados; no se asegura un checkout MVC completo solo porque las operaciones backend estén disponibles.

Para regenerar desde la raíz interna: `python Documentacion/tools/generar_api.py`. Requiere reportlab en el intérprete. Produce `Documentacion/API.md` y `output/pdf/Documentacion-de-API.pdf`. Revisar las notas y ejemplos junto al código antes de regenerar. El PDF contiene marcadores de navegación por sección.